Maastricht University

Master Quantum Research Project Data Science & AI

Semester 1 2025/2026

Investigating the Schmidhuber et al. algorithm for planted noisy k-xor

Group 3

Andres Lebbos Habchi	I6377642
Daphne Janissen	I6294745
Julian Klitzke	I6408800
Mathias Yap	I6192890

Department of Advanced Computing Sciences

Supervisors:
Dr. David Mestel
Dr. Georgios Stamoulis

May 1, 2026

Abstract

This quantum research project investigates the quartic-speedup quantum algorithm proposed by Schmidhuber et al. for the planted noisy k-XOR problem by turning the paper's high-level theoretical description into an experimental pipeline. A synthetic dataset of planted and random k-XOR instances (846 in total) was generated with parameter ranges ($12 \leq n \leq 30$ and $m < 255$) that remain feasible for classical simulation, while acknowledging that the parameters are outside the asymptotic regime where the Alice Theorem guarantees are expected to be tight. The classical Kikuchi baseline for distinguishing planted and random instances was implemented and extended with a key recovery part, which converts the spectral information into an explicit estimate of the planted key. The recovery quality follows the expected dependence on the planted correlation ρ across the dataset , becoming perfect in the deterministic regime and typically near-perfect at high ρ and with broader error distributions at lower ρ. For the quantum algorithm, we provided a resource estimation for the guiding state preparation and the main subroutines of the algorithm including Hamiltonian Simulation, Quantum Phase Estimation and Amplitude Amplification and evaluated practical feasibility under classical simulation constraints. Based on the results, the guiding state dominates the cost because its runtime and memory requirements appear to grow exponentially in n when simulated classically, preventing large-scale execution of the full quantum pipeline beyond very small problem instances. Overall, the results should be read as a feasibility study because the full pipeline can be specified and the classical components are evaluated systematically but the quantum pipeline where the claimed quartic speed-up appears is not feasible with the available simulation resources.

1 Introduction

In the field of Quantum Computing, existing quantum algorithms typically fall into two categories; they either achieve an exponential or a quadratic speedup over the classical method. Exponential speedups, such as Shor's algorithm for integer factorization [8], are exceptionally rare. Conversely, quadratic speedups, like Grover's Search algorithm for unstructured search [3], are more common but often do not result in a practical quantum advantage due to overheads with quantum error correction [2, 4]. Recent research indicates that an intermediate regime of super-quadratic polynomial quantum speedups are potentially more feasible for near-term quantum computing. Specifically, quartic speedups are among the most realistic candidates to achieve measurable quantum advantages [2].

1.1 State of the Art

Recent work by Schmidhuber et al. presents an algorithm with a quartic speedup over the best known classical result. They describe a quantum algorithm for the Planted Noisy k-XOR problem, in which a planted signal is embedded within random noise. The algorithm combines Hamiltonian simulation, the Kikuchi method, Quantum Phase Estimation, Amplitude Amplification, and a guiding state to distinguish between the presence of a hidden planted signal or purely random noise. The source of the quartic speedup is due to the Amplitude Amplification and the guiding state [7], and it is the first known instance of this speedup for a broad class of average-case inference problems. Even though the algorithm has currently only theoretical implementation, its structure closely resembles challenges encountered in lattice-and error-based cryptography, making it a valuable case study for assessing potential quantum advantages in cryptanalysis.

1.2 Open Issues and Knowlegde Gaps

Despite the theoretical breakthrough of the quartic speedup algorithm, there are still many knowledge gaps on the impact on Post Quantum Cryptography(PQC). Theoretically, it provides a quartic speedup, but no practical simulation or experimental validation have been performed yet [7]. Currently, there is no concrete resource estimation for implementing the algorithm on quantum hardware. To assess whether the algorithm can practically achieve a quantum advantage, a resource efficient prototype simulated on a small scale on a classical computer is required due to the current limitation of quantum hardware.

Furthermore,the proposed algorithm of Schmidhuber et al. is a decision algorithm to distinguish between a planted hidden key or purely random noise. In order to compromise cryptographic schemes, an extraction or key recovery algorithm is necessary. The transition from a distinguishing algorithm to a key retrieval algorithm has not been explored so far.

Finally, the algorithm is designed for Planted Noisy k-XOR problems, which share structural similarities with cryptographic problems such as Learning With Errors (LWE), but lack the dense lattice and matrix structures characteristic of those schemes. Currently, it is unknown whether quantum speedup can be generalized or adapted

to effectively target these types of denser cryptographic schemes.

1.3 Objectives

This project aims to build on the theoretical work of Schmidhuber et al. by analyzing how their algorithm can be applied and evaluated in practice by focusing on three main objectives.

1.3.1 Optimal Adaptation for Key Recovery

The quartic speed-up algorithm of Schmidhuber et al. is a decision algorithm that certifies whether a given instance is drawn from a planted or random distribution by verifying if the Kikuchi matrix has an eigenvalue above a certain threshold. The first objective is to move beyond the distinguishing task and incorporate key recovery into the algorithm. The recovery of this key is an important step in cryptography, as it transforms a theoretical distinction between random and structured data into a practical compromise of the underlying secret.

1.3.2 Experimental Evaluation

The second objective is to test the adapted algorithm on a small scale by creating a prototype implementation using a quantum computing framework such as Pennylane. The 2 main goals of this experimental part of the project is to validate the key components of the algorithm and resource estimation. These results will provide early insights into the practical feasibility and potential quantum advantage of the algorithm.

1.3.3 Evaluation of Applicability to Real-World Cryptography

The final objective is to assess the real-world threat to cryptographic security posed by the quartic speedup. While the paper of Schmidhuber et al. focuses on the Planted Noisy k-XOR, modern post-quantum schemes such as Kyber (FIPS 203) rely on Learning With Error (LWE) and involve dense matrix and lattice structures [6]. This objective evaluates whether the adapted algorithm for key-recovery can be applied directly to sparse cryptographic primitives, and be generalized or modified to retain its quantum speedup when applied on denser, more complex instances typical of LWE. This analysis will determine whether cryposystems based on related problems must increase their security parameters to defend against potential quantum algorithm attacks.

1.4 Research Questions

Building on the motivation outlined above, this research focuses on the recently proposed Schmidhuber et al. quartic speed-up algorithm. Although this algorithm demonstrated a major theoretical advance in quantum interference, the practical implications and impact on cryptographic security remain unexplored. The goal of this project is to optimally adapt the quartic speed-up algorithm for key recovery, to experimentally evaluate it on a small scale, and to evaluate its applicability to real-world cryptographic settings. The following research questions guide this project.

- How can the quartic-speed algorithm of Schmidhuber et al. be optimally adapted from a distinguishing task to a recovery task to extract the key from a given instance?
- How can the adapted algorithm be implemented as a quantum circuit for small-scale problem instances in Pennylane, and what are the resource requirements?
- To what extent can the algorithm proposed by Schmidhuber et al. or its adaptations be applied to cryptographic problems such as Learning With Errors?

2 Background

This section introduces the problem setting and the theoretical tool used in this research project. First, planted interference problems are discussed where the goal is to decide whether the input data contains a hidden key or is completely random with the focus on planted noisy k-XOR problems. Currently, the best known classical method to solve these problems is the Kikuchi Method [7]. This method solves planted interference problems by building a matrix from the instance and finding its largest eigenvalue since the hidden key produces a higher eigenvalue than random instances. For $k = 2$ this eigenvalue connects directly to the spectral properties of an edge-signed graph. For general k, Kikuchi lifting transforms the higher-order constraints into a larger lifted graph where the same spectral ideas apply. Finally, the key elements of the quantum algorithm proposed by Schmidhuber et al. is discussed. Since the decision between planted and random instances is reduced to estimating the largest eigenvalue, Quantum Phase Estimation (QPE) together with a guiding state, Hamiltonian simulation and amplitude amplification are used to aim for a quartic speedup over classical spectral methods.

2.1 Planted Inference: Planted Noisy k-XOR

Planted inference refers to a class of computational problems in which a hidden structure, the planted signal, is embedded within random noise. The input is typically a random instance that either contains this planted structure or is purely random, and the algorithm must decide which is the case. These problems are motivated by a fundamental question: how can one reliably uncover structure hidden inside randomness? They offer a way to study this question without distractions from real-world noise or modeling assumptions: the randomness is fully specified, the planted structure is known in principle, and the challenge lies entirely in computation.

Examples of planted inference problems include the Planted Clique problem, where a fully connected subgraph is hidden within a random graph, Tensor Principal Component Analysis (Tensor PCA), where a low-rank tensor signal is buried in Gaussian noise, and the Planted Noisy k-XOR problem, also known as Sparse Learning Parity with Noise (Sparse LPN), which involves identifying a hidden bitstring from noisy parity constraints.

The Planted Noisy k-XOR problem involves detecting information from noisy parity constraints generated using a hidden bitstring $z \in \{\pm 1\}^n$. We observe m equations of the form

$$x_{S_i} = \prod_{j \in S_i} z_j \cdot \eta_i,$$

where each $S_i \subset [n]$ is a randomly chosen subset of size k, and $\eta_i \in \{\pm 1\}$ is a noise variable that flips sign with some probability. The product defines a noisy parity k-XOR constraint. Given $x_{S_1}, \ldots, x_{S_m}$, the decision problem is to determine whether these constraints are generated from a planted z or drawn uniformly at random. When the noise rate is high or the number of constraints is small relative to n, the correlation with the planted solution is weak, making the problem computationally hard.

What makes planted inference especially interesting is the computational-to-statistical gap. In many parameter regimes, the planted signal is strong enough to be statistically detectable. In theory, the planted signal could be recovered with unlimited computation, but no known efficient algorithm can do so in practice. This gap highlights fundamental limitations in algorithmic techniques.

2.2 Kikuchi Method

We consider the *decision version* of the planted noisy k-XOR problem where we must distinguish between the *planted model* and randomly generated *null model*. For the planted case, $\eta \in (0, 1/2]$ is the *noise rate* at which the result of an XOR-Clause is flipped to be inconsistent with the planted assignment. We introduce ρ:

$$\rho = 1 - 2\eta.$$

In the planted model, z satisfies each clause with probability $(1 + \rho)/2$.

2.2.1 Advantage

For any assignment $x \in \{\pm 1\}^n$, we define its *advantage* as the difference between the probability of satisfying and violating a random clause from the instance:

$$\begin{aligned}\mathrm{Adv}(x) = &\Pr[\text{clause satisfied by } x] \\ &- \Pr[\text{clause violated by } x].\end{aligned}$$

In a random instance drawn from the null model, with high probability no assignment achieves an advantage significantly larger than $O(1/\sqrt{m}) \simeq 0$ when the number of clauses m is large compared to n. In contrast, in a planted instance the planted assignment z satisfies each clause with bias ρ, and

$$\mathrm{Adv}(z) \simeq \rho.$$

This bias provides a detectable signal whenever ρ is sufficiently large relative to the fluctuations induced by random constraints.

2.2.2 2-XOR: Edge-Signed Graphs and Spectral Bounds.

When $k = 2$, each XOR clause involves a pair of variables (i, j) and can be represented as an *edge-signed graph*: vertices correspond to variables, edges encode XOR relations, and each edge carries a sign $G_{ij} \in \{\pm 1\}$ indicating whether the XOR constraint enforces agreement or disagreement. Let $G \in \mathbb{R}^{n \times n}$ denote the signed adjacency matrix of this graph, and let $\bar{d}$ denote its average degree. For a normalized assignment vector x with entries $x_i \in \{\pm 1/\sqrt{n}\}$, the advantage can be written as

$$\mathrm{Adv}(x) = \frac{1}{\bar{d}} x^\top G x,$$

and we can find an upper bound on the advantage that relates to the eigenvalues of G:

$$\mathrm{Adv}(x) \leq \frac{2\,\lambda_{\max}(G)}{\bar{d}},$$

where $\lambda_{\max}(G)$ denotes the largest eigenvalue of G.

2.2.3 Upper Bound on Eigenvalues under Null and Planted Models.

The behavior of $\lambda_{\max}(G)$ differs sharply between the null and planted models. **Null model.** When the edge signs are uniformly random ($\rho = 0$), the matrix G resembles a random symmetric matrix with i.i.d. Rademacher entries on its support. Classical results in random matrix theory imply

$$\lambda_{\max}(G) \lesssim C\sqrt{\bar{d}},$$

with $C = O(1)$ or $O(\sqrt{\log n})$ depending on the degree distribution. **Planted model.** If the instance is generated from a planted assignment with advantage ρ, then we expect the planted assignment z to (approximately) achieve the maximal advantage:

$$\rho \approx \mathrm{Adv}(z) \approx \mathrm{Opt} := \max_{x\in\{\pm1\}^n} \mathrm{Adv}(x).$$

Directly computing Opt is NP-hard, but for 2-XOR one can consider the continuous relaxation given by the top eigenvalue of G. Letting $m = |E|$ be the number of edges and $\bar{d} = 2m/n$ the average degree, the relaxation satisfies

$$\lambda_{\max}(G) = \max_{y\in\mathbb{R}^n} \frac{y^\top G y}{y^\top y} \geq \frac{2m}{n}\cdot \mathrm{Opt} = \bar{d}\cdot \mathrm{Opt}.$$

Since $\mathrm{Opt} \approx \rho$ under the planted model,

$$\lambda_{\max}(G) \gtrsim \bar{d}\,\rho.$$

Thus, the planted structure induces a rank-one spike in the spectrum with magnitude $\Theta(\bar{d}\,\rho)$, well separated from the $O(\sqrt{\bar{d}})$ noise scale of the null model once $\bar{d} \gg 1/\rho^2$. This gives a simple test:

`accept planted if` $\lambda_{\max}(G) > \tau$,

for a suitable threshold τ chosen between the noise and signal scales.

2.2.4 Beyond $k = 2$: Kikuchi Lifting.

For general k, XOR clauses involve higher-order interactions, and the adjacency matrix representation no longer captures the planted signal as a simple single eigenvalue. To address this, one uses *Kikuchi lifting*, which embeds the problem in a higher-dimensional space. We can think of this as transforming the n-dimensional problem into the 2-dimensional version we have shown how to solve above. The idea is to introduce *region variables* corresponding to subsets of variables of size ℓ ($1 \leq \ell < k$) and construct the *Kikuchi graph* $\mathcal{K}_\ell$: nodes represent region assignments, edges encode pairwise XOR relations derived from the original k-XOR clauses, and the adjacency structure of this graph represents the *Kikuchi Matrix*. Formally, for each clause support $S \in \binom{[n]}{k}$, the Kikuchi lifting constructs a matching

$$K_S^{\ell,n,k} \quad \text{on the vertex set } \binom{[n]}{\ell},$$

where two ℓ-subsets T, U are matched iff $T \triangle U = S$. The lifted graph is obtained by taking the union of these matchings over all clauses.

2.2.5 Example.

Consider a 3-XOR clause over variables (x_i, x_j, x_k) with sign $b = -1$, i.e.

$$x_i \oplus x_j \oplus x_k = 1.$$

Choosing $\ell = 2$, we associate this clause with all $\binom{3}{2} = 3$ pairs of variables $\{(i,j),(i,k),(j,k)\}$. In the lifted $\mathcal{K}_2$ graph, each of these pairs becomes a node, and the clause induces a signed matching between them with sign $b = -1$.

2.2.6 Eigenvalue Bounds for Planted and Null Models.

Let m denote the number of clauses and define $d = \delta_{\ell,n,k}\, m$, where $\delta_{\ell,n,k}$ is the expected degree contribution per clause under Kikuchi lifting. **Planted model.** Under the planted model with advantage ρ, the top eigenvalue satisfies

$$\lambda_{\max}^{\text{planted}}(\mathcal{K}_\ell) \gtrsim \rho\, d,$$

reflecting a rank-one spectral spike aligned with the planted assignment. **Null model.** Under the null model, the signs of the XOR constraints are independent and uniformly random, and there is no underlying planted structure. In this setting, the spectrum of the Kikuchi matrix is governed entirely by random fluctuations. When the *clause density* $\Delta = m/n$ is sufficiently large, classical concentration results for sparse random matrices, combined with the specific structure of Kikuchi matchings, imply that the top eigenvalue remains bounded close to the bulk of the spectrum. More precisely, for any $0 < \kappa \leq 1$ and $0 < \varepsilon \leq \kappa/(2+\kappa)$, if

$$\Delta \geq C_\kappa \cdot \left(\frac{n}{\ell}\right)^{(k-2)/2} \ln n,$$

where ℓ is the degree of the Kikuchi graph, then with probability at least $1 - 3n^{-\varepsilon\ell}$,

$$\lambda_{\max}^{\text{null}}(\mathcal{K}_\ell) \leq \kappa\, d.$$

The value of this C_κ can be hypothetically improved by decreasing it, which would allow for a Kikuchi Graph with a lower degree.

2.2.7 Detection via Spectral Gap.

Combining the two bounds, when $\rho > \kappa$, the planted and null distributions exhibit a spectral gap:

$$\lambda_{\max}^{\text{planted}}(\mathcal{K}_\ell) \gtrsim \rho\, d \quad \text{vs.} \quad \lambda_{\max}^{\text{null}}(\mathcal{K}_\ell) \leq \kappa\, d.$$

Therefore, a simple spectral test

$$\texttt{accept planted if } \lambda_{\max}(\mathcal{K}_\ell) > \tau,$$

for a suitable threshold distinguishes the two models with high probability.

2.3 Quantum Phase Estimation

As described in the section above, the Kikuchi Method distinguishes null and planted models based on the largest eigenvalue of a matrix. Finding such eigenvalues efficiently can be accomplished using quantum phase estimation (QPE). QPE is an algorithm for estimating the phase φ of a unitary operator U acting on eigenstate $|u\rangle$:

$$U\,|u\rangle = e^{2\pi i\varphi}\,|u\rangle$$

The quantum circuit for the overall QPE procedure is shown in Figure 1 and uses two registers. The first register consists of t ancilla qubits initialized in $|0\rangle^{\otimes t}$ and a second register which contains the eigenstate $|u\rangle$. Each ancilla qubit s placed in superposition and then controls a unitary power U^{2^j} acting on $|u\rangle$. After all the controlled operations , an inverse Quantum Fourier Transform ($FT^\dagger$) is applied to the ancilla register. Measuring this register yields a binary estimate of φ accurate to $t - [log(2 + \frac{1}{2\varepsilon}]$ bits with success probability of $1 - \epsilon$. [5]

2.4 The Quantum Algorithm

The proposed quantum algorithm by Schmidhuber et al. detects planted k-XOR instances by estimating the spectrum of a Hamiltonian K_l derived from the Kikuchi Method. In the planted case, K_l has larger eigenvalues than in the random case. This spectral difference is detected via quantum phase estimation (QPE) applied to a guiding state with significant overlap on the large-eigenvalue eigenspaces.

2.4.1 Guiding State Construction

The guiding state is

$$|\Psi\rangle = |\psi\rangle^{\otimes \ell/k},$$

such that

$$|\psi\rangle = \frac{1}{\|\psi\|} \sum_{x\in\{0,1\}^n} \psi(x)\,|x\rangle,$$

with $\psi(x)$ equal to the number of clauses locally satisfied by assignment x.

In the planted case, $\psi(x)$ is larger for assignments x close to the planted solution x^*, so $|\psi\rangle$ concentrates amplitude on these states. The full guiding state $|\Psi\rangle$ amplifies x^*. Note that $|\Psi\rangle$ is not a probability distribution, but rather a construction that creates high overlap towards the planted solution.

Preparation of $|\psi\rangle$ uses amplitude encoding with a circuit which computes $\psi(x)$ classically and sets amplitudes accordingly. The paper assumes efficient implementation or access to quantum memory for this.

2.4.2 Hamiltonian Simulation

To apply phase estimation to the Hamiltonian K_l, one needs a unitary rather than a Hamiltonian. This unitary can be written in the form

$$U = e^{-iK_l\tau}$$

This can be done with Hamiltonian simulation. The simulation is efficient because K_l is constructed as a sum of local, commuting terms (each corresponding to a clause in the k-XOR instance) y.

2.4.3 Phase Estimation

Quantum Phase Estimation (QPE) is applied using the guiding state $|\Psi\rangle$, the unitary U, and its powers

$$U^{2^j} = e^{-iK_l\tau 2^j}$$

After controlled evolutions, the inverse QFT on the phase register yields an eigenvalue estimate, measured with probability proportional to the squared overlap of $|\Psi\rangle$ with the corresponding eigenvector.

2.4.4 Amplitude Amplification

Because the guiding state may only have small but non-negligible overlap with the high-eigenvalue eigenstates in the planted case, the probability of observing such an eigenvalue in a single run of phase estimation may be low. To boost the probability of success, the algorithm uses amplitude amplification, a generalization of Grover's algorithm.

Amplitude amplification increases the probability of obtaining a "good" measurement outcome — in this case, an eigenvalue above a certain threshold — quadratically in the number of repetitions. The algorithm checks whether the estimated eigenvalue is above a planted-vs-random distinguishing threshold and uses amplitude amplification to increase the chance of measuring such a large value.

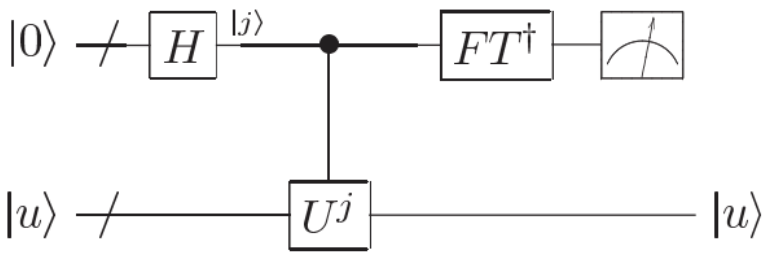

Figure 1: Schematic representation of the overall phase estimation procedure. The top t qubits (the symbol '/' indicates a bundle of wires) form the first register, while the lower qubits constitute the second register, used to implement the unitary operator U. The state $|u\rangle$ is an eigenstate of U with eigenvalue $e^{2\pi i\phi}$. Upon measurement, the output provides an approximation to ϕ, accurate to $t - 2\log_2(1/\epsilon) + 1/2$ bits with probability at least $1 - \epsilon$. Source: [5]

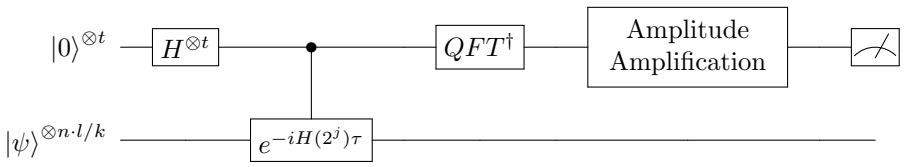

Figure 2: Circuit diagram of the quartic speedup algorithm. The top register (phase register) is initialized in the $|0\rangle^{\otimes t}$ state, transformed by Hadamard gates, and used to control the operator $e^{-iH(2^j)\tau}$. After applying the inverse Quantum Fourier Transform ($QFT^\dagger$), amplitude amplification enhances the success probability. The bottom register prepares the guiding state $|\psi\rangle$ and undergoes controlled operation.

2.4.5 Computational Analysis: Quartic Speedup

The quartic speedup achieved by the quantum algorithm arises from two distinct quadratic improvements:

- **Quadratic Speedup from Guiding State Overlap:** The guiding state $|\Psi\rangle$ is designed to have a significantly higher overlap with the high-eigenvalue eigenvectors. This increases the probability that Quantum Phase Estimation yields eigenvalues indicative of the planted solution by a quadratic factor compared to classical sampling.
- **Quadratic Speedup from Amplitude Amplification:** To compensate for the remaining low success probability of Quantum Phase Estimation measurements, amplitude amplification is applied. This technique provides an additional quadratic speedup by boosting the probability of measuring an eigenvalue that is most likely to go beyond the distinction threshold.

The combination of these two quadratic speedups yields an overall quartic runtime speedup over the classical algorithm.

2.4.6 Assumptions

The algorithm relies on several key assumptions for its correctness and speedup:

- The input is drawn from either a **uniformly random** distribution or a **planted noisy** one, where a hidden assignment is weakly correlated with the observed constraints (through random phase flips).
- There exists a sufficiently large and detectable **spectral gap** between planted and random instances.
- The **Kikuchi matrix** can be represented as a **sparse Hamiltonian** whose nonzero entries are efficiently computable.
- An **efficiently preparable guiding state** exists with inverse-polynomial overlap with the low-energy (planted) subspace of the Hamiltonian.
- The **planted correlation** ρ is above the statistical detectability threshold.
- The algorithm implicitly assumes **QRAM** or **QRAM-like data access** to efficiently compute or query Hamiltonian entries.

3 Methodology

In this section the complete experimental pipeline used in the project is described. We first explain the generation of the synthetic k-XOR dataset consisting of random and planted instances and the parameter selection using the constraints from Alice's theorem to ensure simulation feasibility. After the dataset generation, we introduce a classical and a quantum algorithm to distinguish between a random or planted k-XOR instances. The classical approach is based on the Kikuchi method and the quantum approach is based on the Schmidhuber

et al. quantum algorithm. Finally, a key retrieval algorithm is discussed that converts spectral information into an estimate of the planted key.

3.1 Dataset Generation

A dataset of synthetic k-XOR problems with variables $x_1, x_2, ..., x_n \in \{1\}$ was generated. Each instance consists of m clauses of the form (S_i, b_i) where S_i is a subset of k variables indices and $b_i \in \{1\}$ is the sign of the clause. Throughout this research project 0-based indexing was used, so $[n] = 0, 1, \ldots, n-1$ and each scope satisfies $S_i \subset [n]$ with $|S_i| = k$.The scopes are stored as an integer array $scopes \in \mathbb{N}^{m \times k}$, where each row contains the k indices of a clause, sorted in ascending order. The clause signs are stored in a vector $b \in \pm 1^m$.

Furthermore,each instance contains metadata that describes the sampling distribution. For planted instances, the planted advantage ρ and secret key $z \in \{1\}^m$ are stored, and for random instances these fields remain empty. Finally, all instances are serialized as *.npz* files and include explicit keys for all arrays and metadata flags to ensure instances can be loaded reliably and validated without reliance on implicit conventions.

3.1.1 Instance Distribution

Two types of instances were generated: random instance, which contain purely noise and planted instances which contain a hidden key z.

Random Distribution Random instances are generated by first uniformly sampling each scope S_i at random from the k-subsets of $[n]$ without replacement. Then, each clause sign b_i is sampled independently from a Rademacher distribution, $\Pr(b_i = +1) = \Pr(b_i = -1) = 1/2$, resulting in instances without any planted structure.

Planted Distribution Planted instances are generated by first sampling a secret key $z \sim Unif(\{1\}^n)$. The scopes S_i are sampled identically to the random instances and for each clause an independent noise variable $\eta_i \in \{1\}$ with

$$Pr(\eta_i = +1) = \frac{1+\rho}{2}, Pr(\eta_i = -1) = \frac{1-\rho}{2}$$

so that $\mathbb{E}[\eta_i] = \rho$ is computed. Next, the product of the planted assignment on the clause scope is computed

$$z_{S_i} = \prod_{j \in S_i} z_j$$

and the clause sign is defined as

$$b_i = \eta_i \cdot z_{S_i}$$

. The final dataset contains instances with different noise levels by going over a fixed grid with intervals of 0.2.

3.1.2 Parameter Selection

The parameters (n, k, m) were not chosen arbitrarily. Problem sizes must be feasible for simulation and to apply the Kikuchi method, the parameters must satisfy the constraints of Alice Theorem from [7] shown in Theorem 1. Therefore, a brute-force search over possible parameters $(n, k, \ell, \epsilon, \kappa)$ was performed and to select a suitable number of clauses m the theorem's density requirement was applied.

Different values of n, k, ℓ, ϵ and κ were tested on a fixed grid, while the constraints of the Alice theorem were satisfied. For all the values of n, k, ℓ, ϵ and κ that met these constraints, the clause density $\Delta = \frac{m}{n}$ and the threshold were computed. The smallest m was efficiently selected from a fixed m grid that satisfies $\Delta \geq threshold$ using a binary search over a sorted list of eligible m values. Finally, for all feasible sets, the theorem's failure probability bound was evaluated to ensure theoretical relevance. The parameters $n, m, k, \ell, \kappa, \epsilon$, $\Delta = m/n$, $threshold$, the ratio $\frac{n}{k\ell}$ and the failure probability found by the brute-force search were serialized to a pickle file named *results.pkl*.

3.1.3 Dataset Generation Procedure

The final dataset of k-XOR instances was constructed in two stages. First, the *results.pkl* file, containing all the instances that satisfy the Alice theorem, were filtered to ensure the instances remained feasible for simulation and compatible with our implementation constraints and retained only configurations that satisfy.

$$n < 31, \qquad m < 255, \qquad \text{fail_bound} < 0.1.$$

Because our simulations are limited to small problem sizes, the asymptotic condition in Alice's Theorem $n >> k\ell$ cannot be satisfied strictly. Therefore, in our final dataset we use the relaxed constraint $n \geq n_{factor} k\ell$ with $1 \leq n_{factor} < 4$.

Second, for the 141 remaining configurations of n, k, and m, instances of different noise levels were generated on a fixed grid with step size 0.2 $\rho \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$. A random baseline for every instance was also added by setting $\rho = 0.0$ and resulted in a final dataset with 846 instances. Each of the generated instances were saved as a compressed *npz*. file containing the clause scopes, clause signs, ρ value used and whether an instance is random or planted. Filenames encode the generating parameters to support traceability. To ensure reproducibility of the dataset generation process throughout the

whole pipeline random number generator from NumPy was used with a fixed seed. All the *npz*. files contain all arrays and metadata required enabling deterministic regeneration and reliable loading and validation.

3.2 The Algorithm

This subsection introduces the classical and quantum algorithm implemented in this research project to distinguish random from planted k-XOR instances. The classical algorithm based on the Kikuchi method is discussed first. For a given instances it constructs the Kikuchi matrix and computes its largest eigenvalue. To classify the instances, the computed eigenvalue is compared to a threshold. Next, the quantum pipeline based on the Schmidhuber et al. quantum algorithm is discussed. This algorithm targets the same spectral signal but uses Hamiltonian simulation, QPE supported by a guiding state and amplitude amplification instead.

3.2.1 Classical

As a baseline, we implemented a classical algorithm based on the Kikuchi method. The algorithmic pipeline consists of three main components. First, a verification routine determines whether a given problem instance satisfies the conditions of a specified Alice theorem. Second, an efficient procedure constructs the corresponding Kikuchi matrix for the selected parameters and stores it in an appropriate sparse matrix representation. Third, an eigenvalue solver is employed, operating in two distinct modes depending on the application context.

In the standard classification mode, the solver computes the dominant eigenvalue of the Kikuchi matrix using the classical power iteration method. Power iteration converges to a leading eigenvector, from which the associated eigenvalue is estimated. If this eigenvalue exceeds a previously determined threshold, the instance is classified as planted. If it is not, it is classified as random. In addition to this basic mode, a second mode is implemented for the evaluation of the guiding state, which is discussed in the subsequent section. This mode combines an a priori upper bound on the number of relevant eigenvalues with a dedicated eigensolver library implementing the implicitly restarted Arnoldi method, allowing for more robust spectral analysis.

As discussed briefly in the section on problem set generation, our approach relies on the concept of an *Alice theorem*. In particular, we adopt the bounds presented by [7], stated as Theorem 23, which we reproduce here for completeness.

Verification Routine For purposes of verification, we employ the Alice Theorem as described by [7]. As previously described, while some of these parameters are described by inherent properties, variables such as the Kikuchi lift ℓ and parameters κ and ε must be inferred at runtime. After computing the parameters, this routine outputs a threshold value λ_{max}.

Theorem 1 (Alice Theorem)**.** *Let $\ell, n, k \in \mathbb{N}^+$ with k even, $\ell \geq k/2$, and $n \gg k\ell$. Let $\kappa \leq 1$ and $0 < \varepsilon \leq \frac{\kappa}{2+\kappa}$, and assume that m satisfies*

$$\Delta := \frac{m}{n} \geq C_\kappa \cdot (n/\ell)^{(k-2)/2}, \text{ where}$$

$$C_\kappa = \frac{2(1+\varepsilon)(1+\kappa)}{\kappa^2}\binom{k}{k/2}^{-1} \cdot \ln n \tag{1}$$

Then for $\mathcal{I}$ drawn from either $R_{n,k}(m)$ or $\widetilde{R}_{n,k}(m)$, except with probability at most $3n^{-\varepsilon\ell}$ we have

$$\lambda_{\max}(K_\ell(\mathcal{I})) \leq \kappa d, \text{ where}$$
$$d = \delta_{\ell,n,k} \cdot m \tag{2}$$

Kikuchi Matrix Computation Given a k-XOR instance on n variables and a fixed Kikuchi lift parameter ℓ, we implemented a simple procedure for constructing the corresponding Kikuchi matrix. The matrix is indexed by ℓ-subsets of $[n]$ and is stored in sparse form.

Algorithm.

1. Enumerate all subsets in $\binom{[n]}{\ell}$ and fix an ordering
$$\binom{[n]}{\ell} = \{T_1, \ldots, T_N\}, \quad N = \binom{n}{\ell}.$$
Store a lookup structure that maps each subset to its index.

2. Initialize an $N \times N$ sparse matrix K with all entries equal to zero.

3. For each ℓ-subset T_i and each clause (S_a, b_a) of the k-XOR instance, compute the symmetric difference
$$U = T_i \triangle S_a.$$
If U is again an ℓ-subset, let $T_j = U$ and set
$$K_{i,j} = K_{j,i} = b_a.$$

The resulting matrix is symmetric by construction and has at most one nonzero entry per clause for each row. For each fixed row indexed by T_i, at most one matrix entry is generated per clause, and only when the symmetric difference preserves cardinality. Consequently, the number of nonzero entries in each row is bounded by the number of clauses, which is polynomial in n. Since the matrix has $N = \binom{n}{\ell}$ rows, the total number of nonzero entries is at most

$$O\left(|\mathcal{C}| \cdot \binom{n}{\ell}\right),$$

Eigenvalue Solver To solve the problem classically, we compute the largest eigenvalue of the Kikuchi matrix K_ℓ and compare it against the prescribed threshold. In addition, we use this computation to evaluate the quality of the guiding state preparation and to study the behavior of the spectral gap as a function of the problem parameters. For both purposes, we employ the implicitly restarted Arnoldi method, as implemented in **ARPACK** [1].

3.2.2 Quantum

This section describes the quantum version of the Schmidhuber et al. algorithm as implemented in our experiments. We first restate the ideal quantum algorithm, then explain how each component was instantiated in practice, with particular emphasis on the guiding state construction, the Kikuchi matrix Hamiltonian, and the application of phase estimation and amplitude amplification. Finally, in the results we discuss the limitations observed in the implementation, including a comparison with a classically computed guiding state and an analysis of resource requirements.

The Quantum Algorithm The goal of the quantum algorithm is to distinguish planted from random k-XOR instances by estimating a large eigenvalue of the Kikuchi matrix K_ℓ. In the planted case, K_ℓ exhibits a spectral spike of magnitude $\Theta(\rho d)$, while under the null model its spectrum remains bounded by κd. The algorithm proceeds as follows:

1. Prepare a guiding state $|\Psi\rangle$ that has non-negligible overlap with the top-eigenvalue eigenspace of K_ℓ in the planted case.

2. Simulate the unitary $U = e^{-iK_\ell \tau}$ from the hermitian K_ℓ using Hamiltonian simulation.

3. Apply Quantum Phase Estimation (QPE) using $|\Psi\rangle$ as input to obtain an estimate of an eigenvalue of K_ℓ.

4. Use a threshold test to mark outcomes whose estimated eigenvalue exceeds the planted-vs-random threshold.

5. Apply amplitude amplification (Grover-style iterations) to boost the probability of observing good outcomes which are marked through the planted-vs-random threshold test.

Where a measurement yielding an eigenvalue above the threshold is taken as evidence for a planted instance.

Guiding State Implementation The theoretical algorithm assumes access to a guiding state of the form

$$|\Psi\rangle = |\psi\rangle^{\otimes \ell/k}, \qquad |\psi\rangle = \frac{1}{\|\psi\|} \sum_{x\in\{\pm 1\}^n} \psi(x)\,|x\rangle,$$

where $\psi(x)$ counts the number of clauses locally satisfied by the assignment x. In the planted case, $\psi(x)$ is maximized near the planted assignment z, resulting in increased overlap with the leading eigenspace of K_ℓ.

The exact amplitude encoding of $\psi(x)$ is expensive to compute due to the exponential size of the assignment space. Therefore, we rely heavily on the power of quantum computers to build $|\Psi\rangle$. A quantum state for a clause is represented by the amount of qubits equal to the amount of variables. For a given clause, we first select one variable in its scope to act as a reference qubit. Hadamard gates are applied to all other qubits in the clause, placing them in an equal superposition. This implies that the reference qubit is predetermined by the values of all other assignments and the clause. Therefore, we use a sequence of controlled-X gates conditioned on the values of all other variables in the clause. This procedure creates an equal superposition over all assignments that satisfy the selected clause. Figure shows an example on how to build two particular clauses.

To incorporate multiple clauses, additional ancilla qubits are introduced to prepare an equal superposition over clause indices. Controlled-SWAP operations are then applied, conditioned on the ancilla register, such that the clause-specific superposition state corresponding to the selected index is loaded into the main register. The resulting guiding state is therefore a uniform superposition over clause-wise satisfying subspaces.

Classical Computation of the Guiding State Unfortunately, we do not have access to a quantum computer, so the quantum circuits needed to be simulated classically. This simulation is efficient to implement for small problem sizes. As a result, as the number of variables grew, it became computationally unfeasible. Therefore, to evaluate the effectiveness of the guiding state, we computed $\psi(x)$ classically for all assignments x.

Kikuchi Matrix as a Hamiltonian The Kikuchi matrix K_ℓ was constructed explicitly from the instance clauses using the Kikuchi lifting described in Section 3.2. Each clause induces a signed matching on ℓ-subsets, and the resulting matrix is sparse and Hermitian. We convert K_ℓ to a Unitary $U = e^{-iK_\ell \tau}$ by using Hamiltonian Simulation. This procedure can be

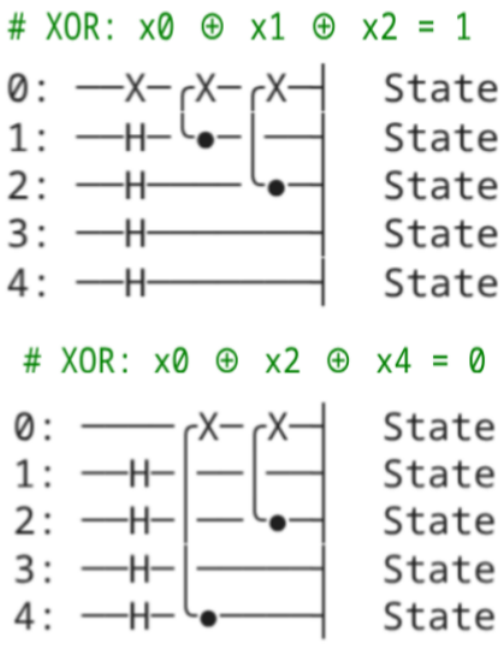

Figure 3: The above image shows how to build a circuit that creates an equal superposition of assignments that are satisfied by a clause using two clauses as examples. These clauses are taken from a problem where $n = 5$ and $k = 3$. We note that x_0 is picked as the reference qubit and every other qubit has a Hadamard applied. Then CNOT gates are applied for every other variable in the clause where the other variable is the control and the reference qubit is the target. Finally, an X gate is applied on the reference qubit only if the right hand side of the clause is a 1.

done because K_ℓ is hermitian. The eigenvectors remain the same but the eigenvalues get transformed appropriately.

Applying Quantum Phase Estimation Quantum Phase Estimation was applied to the unitary $U = e^{-iK_\ell \tau}$, with the guiding state as input. The phase register size was chosen to provide sufficient resolution to distinguish eigenvalues above and below the threshold τ. Phase estimation was done on enough copies of $|\Psi\rangle$ to satisfy the dimension of U.

Amplitude Amplification and Threshold Oracle To compensate for the low success probability of QPE, amplitude amplification was applied. The amplitude amplification is a two step process repeated multiple times. The first is a phase inversion based on an oracle marking good outcomes. The oracle uses the estimated eigenvalues compared to a threshold and marks as good those that are beyond the threshold which separated planted from random instances by defining planted as good. This oracle is implemented as a comparator acting on the phase register. Then, the second part is an inversion around the mean. In theory, amplitude amplification provides a quadratic boost in success probability, contributing to the overall quartic speedup.

3.3 The key retrieval algorithm

The key retrieval algorithm starts from a vector $v_{\text{top}} \in \mathbb{C}^N$ produced earlier in the pipeline. This vector is *not* the key itself. Instead, it is used to construct a voting matrix, from which we then extract an n-dimensional eigenvector and convert it into the final key estimate.

Voting matrix construction. Given v_{in}, we construct the voting matrix V using the common-remainder/ℓ-subset encoding. In practice, only the first $\binom{n}{\ell}$ entries of v_{in} are used, corresponding to the ℓ-subset register. The resulting matrix $V \in \mathbb{C}^{n\times n}$ is Hermitian and is designed so that its top eigenvector contains the key information.

Key recovery from the voting matrix. Next, we compute the top eigenvector of V (or a good approximation of it) and denote the resulting vector by $v \in \mathbb{C}^n$. We considered two implementations: (i) a quantum procedure based on quantum phase estimation (QPE) together with amplitude amplification, and (ii) a classical eigensolver that computes a small number of leading eigenpairs.

From v we form the key estimate by rounding entrywise,

$$\hat{z} \;=\; \operatorname{sign}(\Re(v)) \in \{\pm 1\}^n.$$

Since eigenvectors are only defined up to a global sign, we choose between $\hat{z}$ and $-\hat{z}$ by taking the one with the larger instance advantage. For completeness, we also record the diagnostic index

$$x \;=\; \arg\max_i \, |(v)_i|,$$

which is useful for debugging but is not the recovered key.

Quantum eigenvector extraction on V. In the quantum variant, we run QPE with the voting matrix as Hamiltonian, and use amplitude amplification to bias the outcome towards the top eigenspace. The main points are:

1. **Input state** The system register is initialized in an equal superposition.

2. **Hamiltonian** We encode the voting matrix as a Hamiltonian and run QPE on it. For numerical stability we work with a shifted matrix

$$V' \;=\; V + \alpha I,$$

which leaves the eigenvectors unchanged. If required by the circuit implementation, we embed V' into a power-of-two dimension by zero-padding.

3. **Oracle** The amplitude amplification oracle marks phase-register outcomes associated with the top eigenvalue of V' (up to a small neighborhood of phase indices). This concentrates probability mass on the top eigenspace and yields a system-register vector that we use as v.

Classical eigenvector extraction on V. In the classical variant, we compute leading eigenpairs of V using an iterative eigensolver. As in the quantum case, we may work with $V' = V + \alpha I$; this does not change eigenvectors, and eigenvalues can be shifted back afterwards. We then take the eigenvector corresponding to the largest (unshifted) eigenvalue as v, and apply the same rounding and sign choice to obtain the final key estimate $\hat{z}$.

4 Results

In this section, the results of our experiments on the complete synthetic k-XOR dataset are presented. First, the generated problems sets, and the effect of the parameters sizes on the Kikuchi Matrix and spectral gap are analyzed. For the quantum algorithm a resource estimation of the main quantum subroutines is provided, followed by an empirical study of the guiding-state preparation cost. Finally, the performance of the key-extraction algorithm is evaluated accros different noise levels ρ and other parameters. The main trends in recovery quality and runtime are discussed.

4.1 Problem Set Generation

We begin by examining problem set generation from a theoretical perspective, with particular emphasis on the computational resources required to obtain a nontrivial Kikuchi matrix solution, as emphasized in the original work. The case $k = 2$ is trivial and does not require a Kikuchi lift, while the construction described in the paper applies only to even values of k. Consequently, the smallest nontrivial family of instances corresponds to $k = 4$.

For $k = 4$, the paper provides an illustrative example at advantage $\rho = 0.25$, with suggested parameters $\ell = 32$ and

$$\frac{m}{n} = 0.25 \ln n.$$

The Alice Theorem further requires that $n \gg k\ell$. Even under a significantly relaxed version of this condition, namely $n \geq k\ell$, the smallest admissible instance would have $n = 4 \cdot 32 = 128$. For these parameters, the resulting Kikuchi matrix has dimension on the order of

$$\binom{n}{\ell} \approx 1.48 \times 10^{30}.$$

Even assuming extreme sparsity, constructing such a matrix would require time on the order of $O(n^\ell)$, followed by an eigenvalue computation involving a comparable number of nonzero entries. This places the direct application of the theorem far beyond any realistic computational regime.

For this reason, we instead consider smaller problem instances that explicitly violate the domination constraint of the Alice Theorem. While these instances fall outside the regime in which the theorem provides formal guarantees, they allow us to empirically study the behavior of the spectral gap and its dependence on the underlying parameters.

Evaluation Procedure To evaluate generated problem instances, we compute the dominant eigenvalue of the Kikuchi matrix constructed from the problem ensemble used to generate the guiding state. Figure 4 shows the dominant eigenvalue alongside the theoretical threshold for a range of Kikuchi matrix sizes and advantage values ρ.

As shown in the figure, small instances may formally satisfy the algebraic conditions of the Alice Theorem yet still fail to produce eigenvalues exceeding the prescribed threshold. This effect persists even at $\rho = 1$, corresponding to fully deterministic problem generation. The underlying reason is that, for small matrix dimensions, the Kikuchi matrix does not exhibit a sufficiently well-separated cutoff eigenspace, despite satisfying the other parameter constraints.

To better quantify this behavior, we examine the gap between the dominant eigenvalue and the threshold, normalized by the threshold value. The normalized gap,

$$\frac{\lambda_{\max} - \text{threshold}}{\text{threshold}},$$

is shown in Fig. 5 for a range of matrix sizes and advantage values. For small matrix dimensions, the normalized gap is predominantly negative, indicating that the dominant eigenvalue remains below the theoretical threshold. As the Kikuchi matrix dimension increases, a clear trend emerges: higher advantage values lead to faster growth of the dominant eigenvalue, eventually closing and surpassing the threshold.

This behavior is consistent with the derivation underlying the Alice Theorem, in which the spectral gap depends on the average degree d of the Kikuchi matrix, the constant κ, and

the advantage ρ. Increasing the matrix dimension increases the average degree d, thereby enlarging the spectral gap, with the effect becoming more pronounced at larger advantage values. We note that, despite the strictness of the theoretical threshold, the dominant eigenvalue for genuinely random instances remains well below its maximal observed value. In practice, the eigenvalues of random problem instances do not approach the threshold even approximately. Consequently, modestly relaxing the threshold would substantially improve detection accuracy on our dataset without introducing false positives. This observation suggests that tighter, instance-size–dependent bounds may exist for small problem regimes, beyond those provided by the asymptotic analysis underlying the Alice Theorem.

Computational Cost Finally, we examine the classical computational resources required to construct and analyze the Kikuchi matrix. Figure 7 shows the time required to generate the Kikuchi matrix as a function of its dimension, while Fig. 8 shows the corresponding time required to compute the dominant eigenvalue.

In both cases, we observe a clear log-linear relationship between the Kikuchi matrix size and the computation time. This scaling is consistent with the growth in the number of nonzero entries in the matrix, which increases rapidly with both n and ℓ. Together, these results reinforce the conclusion that, even well below the theoretical parameter regime of the Alice Theorem, the computational cost of Kikuchi-based analysis grows quickly, imposing practical limits on the size of instances that can be studied numerically.

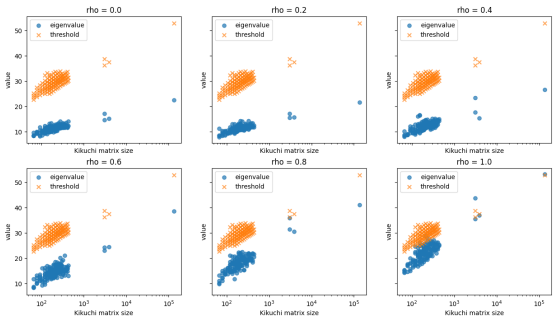

Figure 4: Eigenvalue and threshold values separated by planted advantage rho and compared over Kikuchi matrix size.

4.2 Quantum Resource Estimation

The preparation of the guiding state for an n-variable, m-clause instance with k variables per clause requires $(m+1)n + a + 1$ qubits, where

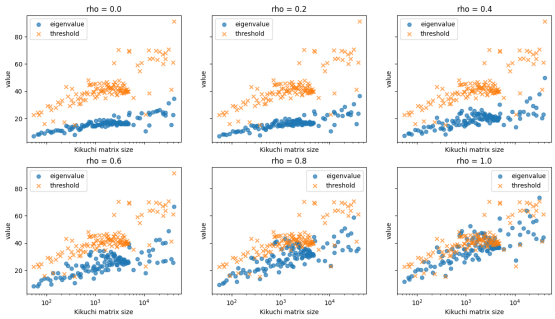

Figure 5: Evaluation of the relation between eigenvalue and threshold values over a range of Kikuchi matrix sizes. The growth of the maximum eigenvalue appears to outpace the growth of the threshold for larger Kikuchi matrix sizes.

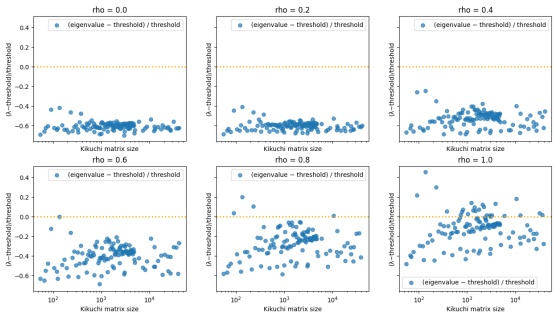

Figure 6: Evaluation of the relation between the normalized difference between threshold and eigenvalue over a range of Kikuchi matrix sizes. For sufficiently large rho, the difference trends towards a 0 difference, which quantifies a succesful identification.

$a = \lceil \log_2 m \rceil + 1$; this accounts for the clause registers, a target register, and ancillary qubits for arithmetic and control operations. In terms of gates, $mn + (m+1)a + m$ Hadamard and Pauli-X gates are needed to generate the clause superpositions and initialize the ancilla, while $(k-1)m + 3mn$ CNOT gates implement the clause entanglement and controlled swaps necessary for encoding of the clauses. Asymptotically, this yields a qubit requirement of $O(mn)$ and a gate complexity of $O(m(n + k + \log m))$. Finally, in order to match the dimension $\binom{n}{\ell} = n^l$ of the kikuchi matrix, at most $O(l)$ copies of the guiding state are needed. The resulting qubit complexity is $O(lmn)$ while the gate complexity is $O(lm(n + k + \log m))$.

A few additional details must be mentioned. First, because the guiding state is designed to have a success probability exceeding 50% given by the ceiling in the ancilla qubits, only a constant number of repetitions (at most two in expectation) are required. Simplifications are possible: the $m+1$ factor in the qubit count can be reduced to two with a shared loading register, and the CNOT count separates naturally into contributions from clause preparation, which scales with k, and controlled-swap opera-

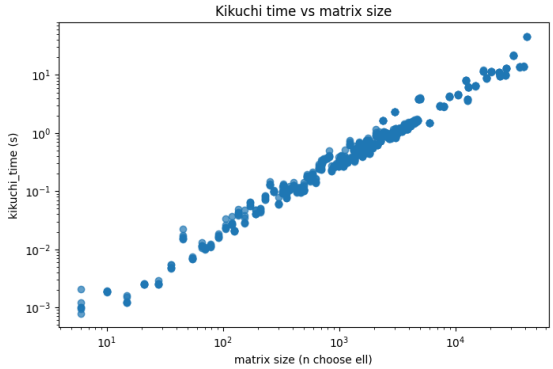

Figure 7: On the y axis: computational time to generate the kikuchi matrix, in seconds. On the x axis, the size of the matrix.

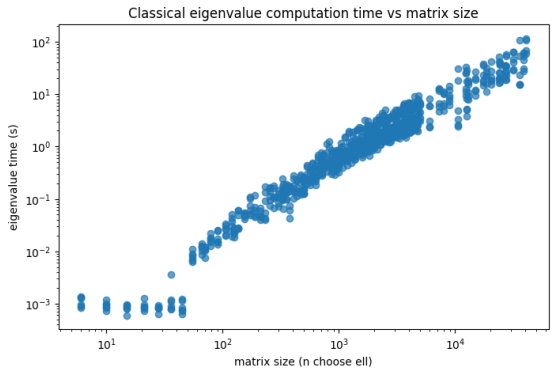

Figure 8: On the y axis: computational time to find a dominant eigenvalue, in seconds. On the x axis, the size of the matrix.

tions, which scale with n.

In addition to the guiding state, other resources are needed for the Hamiltonian Simulation, QPE, and Amplitude Amplification. Considering that the Hamiltonian acts on a space of dimension $\binom{n}{l}$, then representing the system requires $N = \Theta\left(\log \binom{n}{l}\right)$ qubits, which simplifies to $N = \Theta(l \log n)$. In order to resolve eigenvalues to precision ε, QPE employs an additional phase register of $t = \Theta\left(\log \frac{1}{\varepsilon}\right)$ qubits, giving a total qubit count of $\Theta\left(l \log n + \log \frac{1}{\varepsilon}\right)$. The cost of simulating e^{-iHt} scales linearly with $\Theta(n^l)$. A single execution of QPE therefore has gate complexity $\Theta\left(\frac{n^l}{\varepsilon}\right)$, with inverse quantum Fourier transforms and classical control contributing only lower-order overheads.

If amplitude amplification is employed to boost the probability of success, the QPE circuit must be coherently repeated $\Theta\left(\frac{1}{\sqrt{p}}\right)$ times, where p denotes the probability that a single execution of QPE, applied to the chosen initial state, yields an eigenvalue estimate within the desired acceptance window. This results in an overall gate complexity of $\Theta\left(\frac{n^l}{\varepsilon\sqrt{p}}\right)$.

4.3 Guiding-state Preparation

Guiding State Overlap We do not report explicit guiding state overlap computations. Evaluating these overlaps would require computing inner products involving tensor products of already high-dimensional states, which becomes prohibitively expensive at the problem sizes where the spectral guarantees of the Alice Theorem are expected to hold. Conversely, for smaller instances where such overlap computations are tractable, the lifted problems do not exhibit a sufficiently separated cutoff eigenspace, and the assumptions underlying the theorem are not satisfied. As a result, overlap values in this regime are not meaningful indicators of algorithmic performance.

Runtime Guiding-state Preparation The runtime to prepare the guiding-state for the 846 generated k-XOR instances with $12 \leq n \leq 30$ and $m \leq 255$ is displayed in Figure 9 and 10. Both figures show that the preparation time of the guiding-state exponentially increases with the number of variables n. The scatterplot in Figure 10 demonstrates that the runtime increases for fixed values of m and increasing values of n or viceversa. Additionally, the RAM requirements increase exponentially with every every increment of n or doubling of m. These numbers where chosen because they are the maximum executable values with 32GB of RAM, which is the maximum resources at our disposal. The graph in Figure 9 also demonstrates that for

larger values of n the median runtime spans multiple orders of magnitude. The widening spread for larger n indicates that runtime also depends on the specific instance, not only on the parameter values. Overall, these results show that guiding-state preparation becomes a dominant bottleneck as the number of variables n grows, which limits the problem sizes that can be used for simulation.

4.4 Evaluation of Key Extraction

We evaluated key extraction on all 846 test instances using the *classical* eigensolver-based extraction pipeline. The alternative pipeline based on a simulated quantum phase estimation (QPE) circuit was not used for the full retrieval-quality evaluation, since its runtime and memory requirements become prohibitive on limited hardware as the key length n grows.

For each instance, we primarily report (i) the achieved advantage over random guessing of the planted key and (ii), whenever a ground-truth key is available, the Hamming distance (reported as a distance fraction) between the retrieved key and the ground truth. We then summarize these per-instance results using standard descriptive statistics: mean, median, 25% and 75% quartiles (and their interquartile range, IQR), standard deviation, and minimum/maximum values. Results are grouped by the parameters n (key length), ρ, and κ. In addition, ε was varied across runs; we report it separately but keep the focus on n, ρ, and κ.

Figures 11 and 12 show boxplots of the advantage and Hamming-distance fraction for different values of ρ and κ, respectively. The boxplots suggest that changing κ has only a modest effect on typical (median) performance: the median advantage remains similar across the tested values, and the median Hamming distance is 0 for all κ, meaning that at least half of the instances are recovered perfectly. At the same time, larger κ is associated with a wider upper tail: both the maximum Hamming distance and the interquartile range increase as κ approaches 1. This indicates that while the central tendency remains strong, a small number of instances become harder to recover, occasionally even yielding negative advantage (worse than random guessing).

Figure 11 shows a clear and expected dependence on ρ. In our generation setup, $\rho = 0$ corresponds to fully random instances, for which no meaningful ground truth exists, while larger ρ makes instances increasingly consistent with a planted key; in particular, $\rho = 1$ corresponds to the fully deterministic case. Accordingly, extraction is perfect at $\rho = 1$ (advantage = 1 and Hamming distance = 0). Already at $\rho = 0.8$, recovery is essentially perfect in typical cases (median Hamming distance = 0), although rare errors still occur, as reflected by a nonzero maximum Hamming distance. Overall, the key-recovery error decreases sharply as ρ increases, while the advantage increases monotonically, confirming that retrieval quality improves as instances become less noisy and more strongly planted.

For completeness, we additionally ran a limited set of experiments using a quantum-circuit-based key retrieval procedure. Due to computational constraints, these experiments were restricted to instances with key length

$$n \in \{12, 13, 14, 15, 16\}.$$

Since the classical baseline can be evaluated for substantially larger n, this only permits a limited comparison between the circuit-based procedure and the classical eigensolver-based implementation. We therefore refer to Figures 13 and 14 for details.

Across ρ, the qualitative behavior of the circuit-based procedure remains consistent with the classical results: increasing ρ improves recovery, as reflected by higher advantage and lower Hamming distance. However, the corresponding distributions exhibit noticeably larger dispersion and more pronounced asymmetry in several parameter regimes, indicating less stable performance across instances. In particular, some instances still achieve near-baseline performance even at otherwise favorable parameter settings, suggesting occasional failure modes under our limited test set.

A substantially larger sample would be required to draw stronger conclusions about the behavior of the *quantum-circuit-based* retrieval approach in particular. Moreover, on real hardware, repeated finite-shot executions would be necessary to quantify sampling variance. We leave a more detailed analysis to future work due to the computational limitations encountered in this study.

5 Discussion and Conclusion

In this final section, the main results and answers to our research questions are discussed. The practical feasibility of the Schmidhuber et al. inspired pipeline is evaluated, focusing on the simulation limits that restrict the problem size. The main bottlenecks are also highlighted and used to explain their effect on the classical and quantum approaches. Finally, the potential cryptographic impact is evaluated, the main findings of the project are summarized and the most important directions for future work are described.

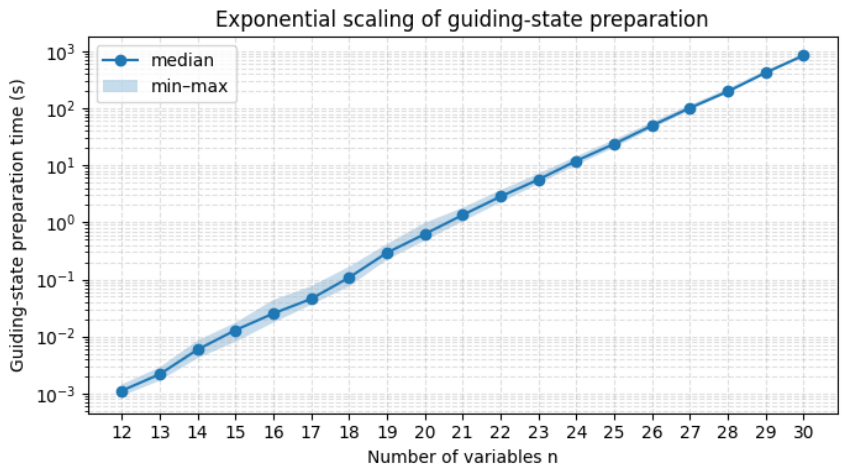

Figure 9: Guiding-state preparation time as a function of the number of variables n with a logarithmic scale on the y-axis. The solid line shows the median runtime across instances for each n, while the shaded region indicates the minimum–maximum range observed.

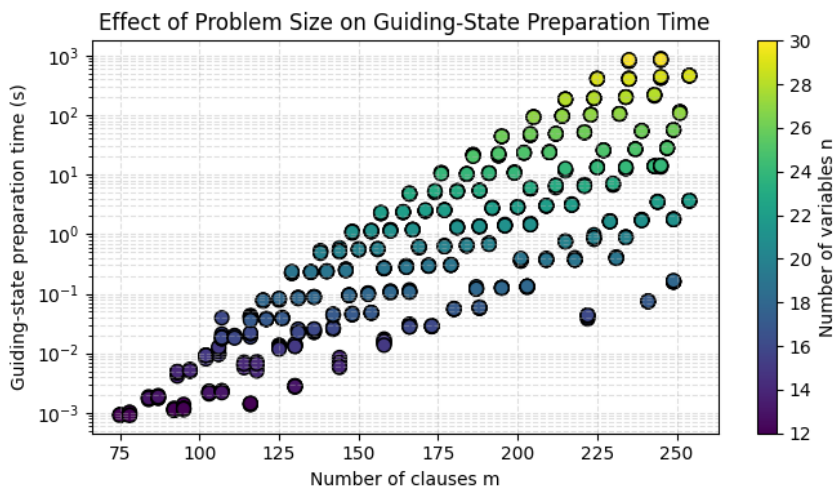

Figure 10: Guiding-state preparation time versus the number of clauses m with a logarithmic scale on the y-axis. Each color indicates the number of variables n. The plot shows that preparation time increases with m and grows substantially for larger n.

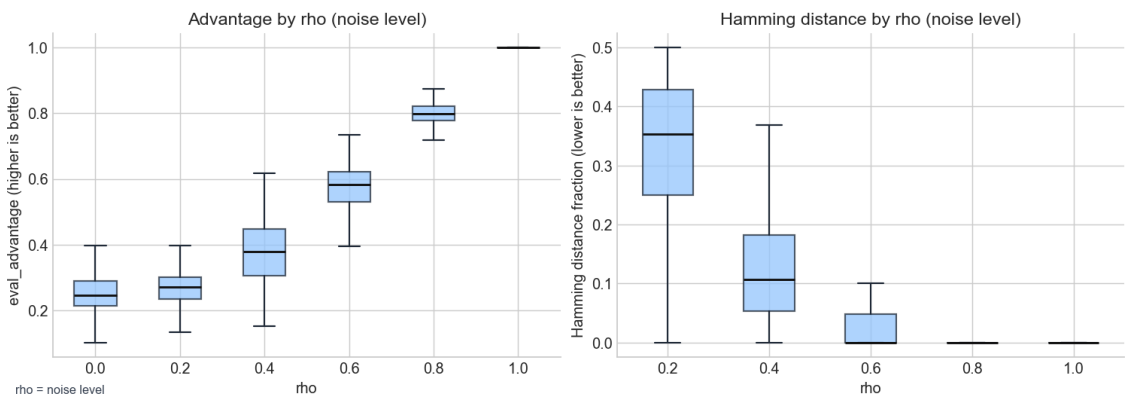

Figure 11: Recovered-key performance as a function of the noise level ρ, when using the **classical** eigensolver for extraction. Left: advantage over random guessing (higher is better). Right: Hamming distance to the planted key (lower is better; shown only for instances where the ground truth key is available). The plots show the median value, minimum & maximum value, as well as the 25% and 75% quantiles.

5.1 Required Resources

As proven in section 4.2, the amount of qubits needed scale polynomial with respect to the in-

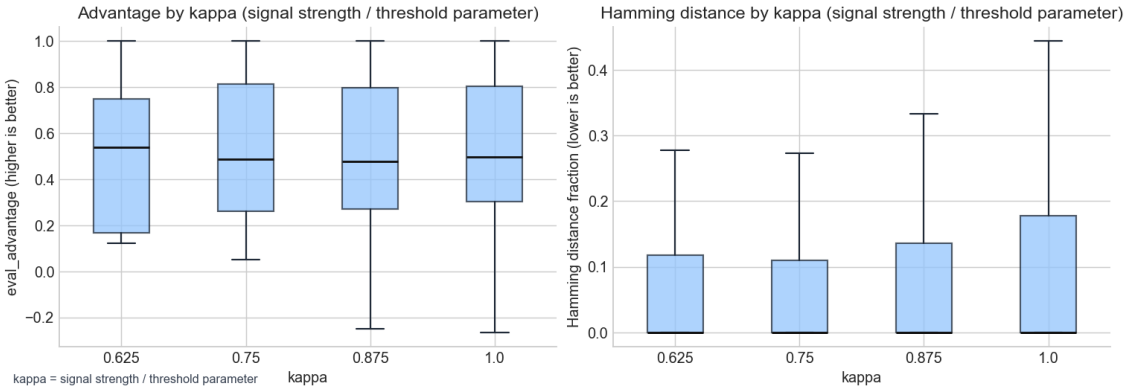

Figure 12: Recovered-key performance as a function of κ (signal strength / threshold parameter), when using the **classical** eigensolver for extraction. Left: advantage over random guessing (higher is better). Right: Hamming distance to the planted key (lower is better; shown only for instances where the ground truth key is available). The plots show the median value, minimum & maximum value, as well as the 25% and 75% quantiles.

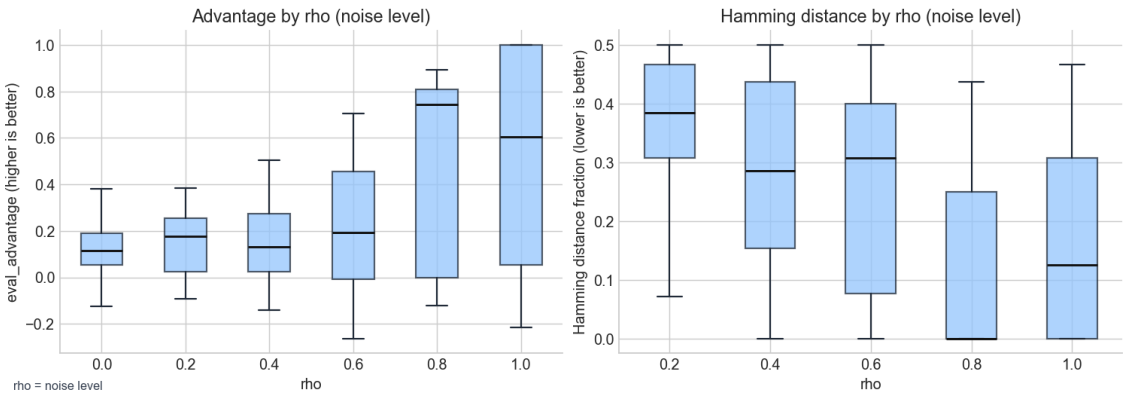

Figure 13: Recovered-key performance as a function of the noise level ρ, when using the **quantum circuit** (QPE & amplitude amplification) for extraction. Left: advantage over random guessing (higher is better). Right: Hamming distance to the planted key (lower is better; shown only for instances where the ground truth key is available). The plots show the median value, minimum & maximum value, as well as the 25% and 75% quantiles.

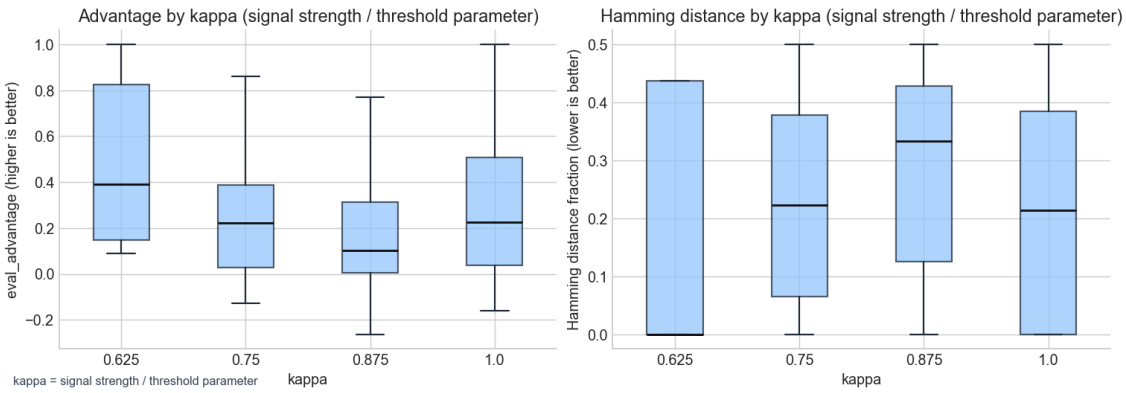

Figure 14: Recovered-key performance as a function of κ (signal strength / threshold parameter), when using the **quantum circuit** (QPE & amplitude amplification) for extraction. Left: advantage over random guessing (higher is better). Right: Hamming distance to the planted key (lower is better; shown only for instances where the ground truth key is available). The plots show the median value, minimum & maximum value, as well as the 25% and 75% quantiles.

puts, particullarly n. However, two simultaneous problems arise. First, per Alice's Theorem, n needs to heavily dominate the other parameters which makes it a large variable. Second, since we don't have a quantum computer at our disposal, in order to simulate these qubits, we

require exponential amount of resources which easily break RAM problems when n is greater than 5.

As an alternative solution, we decided to only compute the guiding state classically for comparison to the eigenvalues of the problems as shown in section 4.3. However, the amount of resources needed classically is exponential when compared to quantum resources, therefore for a modern computer with 32GB RAM, the maximum parameters allowed for the classical simulation where n is at most 30 and m is at most 255. These constraints are given by the array size limit in Python and, more importantly, the amount of RAM available. Unfortunately, increasing n by one or doubling m implies doubling the required RAM to simulate the guiding state. Please note that this is only for the guiding state preparation, running the rest of the Quantum Circuit requires even more resources, and this is the reason why the quantum simulation doesn't run when n is greater than 5. Finally, optimization on the guiding state storage is not a very effective solution given that they are not sparse at all.

In conclusion, we would require a quantum computer with at least hundreds noise-free qubits and even more quantum gates to simulate this algorithm or exponentially growing amounts of memory which is completely unfeasible. Therefore, due to the resource limitations, we strongly believe that security protocols relying on planted inference are safe at least until the quantum barrier is broken.

5.2 Conclusion

This project examined the quartic speedup framework of Schmidhuber et al. [7] algorithm for planted noisy k-XOR with the aim of understanding what the proposal looks like as a concrete pipeline, what can be validated on classical hardware, and where the practical bottlenecks appear. We implemented the classical Kikuchi baseline for distinguishing, translated the quantum pipeline into explicit circuit components, and extended the overall workflow with a recovery procedure that turns spectral information into an explicit key estimate.

Empirically, the classical pipeline is the only part that can be evaluated at scale with the resources available to us. Using a synthetic dataset of 846 instances (filtered to remain feasible for simulation and generated on a fixed grid of noise levels), we evaluated detection and key extraction across parameter regimes and observed the expected dependence on the planted correlation ρ. Recovery is perfect in the deterministic regime and remains essentially perfect in typical cases already at high ρ, while lower ρ yields broader distributions and occasional failure cases. Varying κ has little effect on medians but does affect the tails, which matters when performance is judged not only by typical behavior but also by the presence of hard instances.

In regards to detection problem instances, our empirical investigation highlights a substantial gap between the asymptotic guarantees of the Alice Theorem and the behavior observed in computationally accessible regimes. While the theorem predicts a clear spectral separation for sufficiently large Kikuchi matrices, the matrix dimensions required to formally satisfy its assumptions are far beyond practical reach. For smaller instances, which necessarily violate the domination constraint, the dominant eigenvalue almost always remains below the theoretical threshold even at high advantage, though a clear trend emerges as the matrix dimension increases. At the same time, the computational cost of constructing the Kikuchi matrix and computing its dominant eigenvalue grows rapidly with problem size, imposing strict limits on numerical exploration. These results suggest that more refined, size-aware bounds on the spectral gap may help practical detection for smaller problem instances.

On the quantum side, our main result is a feasibility assessment rather than a large scale performance comparison. While the resource estimates for the ideal algorithm scale polynomially in the relevant input parameters, the simultaneous requirements imposed by the Alice theorem regime and the cost of classical simulation prevent running the full circuit beyond very small problem sizes. In practice, guiding state preparation becomes the dominant bottleneck: both runtime and memory requirements grow exponentially in the parameters when simulated classically, and even when guiding state vectors can be computed for moderate n, simulating the full QPE based pipeline is already out of reach. For completeness we ran a small set of circuit based retrieval experiments (restricted to small key lengths), which qualitatively follows the classical trend with ρ but exhibits much larger dispersion and less stable instance to instance behavior, suggesting that meaningful conclusions would require substantially larger experiments and, on real hardware, careful finite shot analysis.

Overall, these results should be interpreted primarily as an assessment of feasibility. The pipeline can be specified end-to-end, and the classical components can be evaluated systematically on a large benchmark, but the regime in which the claimed advantage becomes meaningful is not accessible to our experiments. The limiting factor is the scalability of the quantum-specific subroutines under classical simulation, most notably guiding-state preparation and the subsequent QPE-based procedure. Under these

constraints, our study does not demonstrate a near-term cryptanalytic impact; instead, it delineates the concrete subroutines and scaling barriers that must be addressed before the quartic-speedup framework can be evaluated in a hardware-realistic setting.

5.3 Future Work

Sparse Hamiltonian Simulation Since our Hamiltionan is large yet sparse, future work could explore simulation techniques that take advantage of this sparsity. Leveraging the structure of the Hamiltonian may allow for more efficient approaches and could enable scaling to larger systems in a way that is not constrained by current technology. Investigating such methods could provide a pathway toward extending our results beyond the current system size.

Small Problem Instances The entirety of the paper relies on Alice's Theorem which clearly defines bounds with regards to heavily dominated planted instances where n highly dominates ℓk. Unfortunately, this generates problem instances that are not currently executable due to high memory constraints or the requirement of a quantum computer. For future work, it would be interesting to define bounds to instances that are not heavily dominated. In other words, expand or add to Alice's Theorem to find bounds for small problem instances which can be simulated and tested on classical computers with reasonable memory.

The Guiding State: A Different Approach The guiding state does not need to be exact. As long as the guiding state has decent overlap with an eigenvector in the region above the threshold, then the quartic speedup should theoretically remain. For future work, we propose the idea of instead of building the guiding states from all the clauses, instead separate the clauses into a two groups: one that is very small and one that is very large. Use the small one for the guiding state computation which heavily reduces the computation complexity, while the large on is used on the Kikuchi hamiltonian. This could theoretically still provide a quartic speedup while potentially fixing the limitations behind the guiding state.

6 Overview Of Who Did What

Please see table 1 for the overview of who did what.

Sec.	Author(s)
Abs.	Daphne
Intro.	Daphne
1.1	Daphne
1.2	Daphne
1.3	Julian
1.4	Daphne
2.1	Andres
2.2	Mathias
2.3	Daphne
2.4	Andres (2.4.1 - 2.4.5)
	Julian (2.4.6)
3.1	Daphne
3.2	Mathias (3.2.1)
	Andres (3.2.2)
3.3	Julian
4.1	Mathias
4.2	Andres
4.3	Mathias
4.4	Julian
5.1	Andres
5.2	Julian
5.3	Andres & Mathias
Flow	Daphne

Table 1: Overview of report sections and author contributions.

References

[1] *1. Introduction to ARPACK*, pages 1–7.

[2] Ryan Babbush, Jarrod R. McClean, Michael Newman, Craig Gidney, Sergio Boixo, and Hartmut Neven. Focus beyond quadratic speedups for error-corrected quantum advantage. *PRX Quantum*, 2(1), March 2021.

[3] Lov K. Grover. A fast quantum mechanical algorithm for database search, 1996.

[4] Torsten Hoefler, Thomas Häner, and Matthias Troyer. Disentangling hype from practicality: On realistically achieving quantum advantage. *Communications of the ACM*, 66:82–87, 04 2023.

[5] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, UK; New York, NY, USA, 10th anniversary edition edition, 2010.

[6] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard (fips 203). Technical Report FIPS 203, U.S. Department of Commerce, National Institute of Standards and Technology, Gaithersburg, MD, USA, August 2024.

[7] Alexander Schmidhuber, Ryan O'Donnell, Robin Kothari, and Ryan Babbush. Quar-

tic quantum speedups for planted inference. *Physical Review X*, 15(2), June 2025.

[8] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997.